

TASK 3 – SECURE CODE ASSESSMENT

Secure Code Assessment of Student Login Application using Python

1. Aim

To examine the source code of a Python application, identify common security weaknesses, and apply secure coding practices.

2. Objective

To identify SQL Injection, insecure password storage, weak input validation and improper error handling, and recommend secure fixes.

3. Application Description

A simple Student Login System developed using Python and SQLite. It supports registration, login authentication and exit.

4. Technology Used

Python, SQLite, password hashing, parameterized SQL queries, input validation and error handling.

5. Source Code

```
import sqlite3
import hashlib

conn = sqlite3.connect("students.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    password TEXT
)
""")
conn.commit()

def register():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if len(username) < 3:
        print("Username must contain at least 3 characters.")
        return

    if len(password) < 8:
        print("Password must contain at least 8 characters.")
        return

    password_hash = hashlib.sha256(password.encode()).hexdigest()

    try:
        cursor.execute(
            "INSERT INTO students (username, password) VALUES (?, ?)",
            (username, password_hash)
        )
        conn.commit()
        print("Registration successful!")
    except sqlite3.Error:
        print("Registration failed.")

def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    password_hash = hashlib.sha256(password.encode()).hexdigest()

    cursor.execute(
        "SELECT * FROM students WHERE username = ? AND password = ?",
        (username, password_hash)
    )

    user = cursor.fetchone()

    if user:
        print("\nLogin successful!")
        print("Welcome,", username)
    else:
        print("\nInvalid username or password.")

def main():
```

```

while True:
    print("\n===== STUDENT LOGIN SYSTEM =====")
    print("1. Register")
    print("2. Login")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        register()
    elif choice == "2":
        login()
    elif choice == "3":
        print("Thank you for using the application.")
        break
    else:
        print("Invalid choice.")

main()
conn.close()

```

6. Sample Output

Command Prompt - Student Login System

```

===== STUDENT LOGIN SYSTEM =====
1. Register
2. Login
3. Exit
Enter your choice: 1

Enter username: student01
Enter password: Student@123
Registration successful!

===== STUDENT LOGIN SYSTEM =====
1. Register
2. Login
3. Exit
Enter your choice: 2

Enter username: student01
Enter password: Student@123

Login successful!
Welcome, student01

===== STUDENT LOGIN SYSTEM =====
1. Register
2. Login
3. Exit
Enter your choice: 3

```

7. Security Assessment

No.	Security Area	Assessment
1	SQL Injection	Protected using parameterized queries.
2	Password Storage	Password is hashed before storage.
3	Input Validation	Username and password length are checked.
4	Error Handling	Database errors are handled with a general message.
5	Authentication	Username and password are verified before login.

8. Vulnerabilities and Recommended Fixes

SQL Injection: Parameterized queries are used instead of string concatenation.

Plain-text Password Storage: Passwords are hashed before storage. For production, use Argon2id, bcrypt or scrypt.

Weak Input Validation: Minimum username and password lengths are checked.

Improper Error Handling: General error messages are shown without exposing internal database details.

9. Secure Coding Practices Used

- Parameterized SQL queries
- Password hashing
- Input validation
- Safe error handling
- Authentication checks

10. Conclusion

The application was reviewed through a manual secure-code assessment. Parameterized SQL queries, password hashing, input validation and safe error handling improve protection against common coding weaknesses.